



Network Intrusion Detection Using Zeek with Grafana Visualization and Discord Alerting

Group 1:

Muhammad Farrel Rachmadya Pasya

Reyandanu Fajri Daniswara

Faculty:

Tri Agus Riyadi

Class:

4CS1

CEP CCIT FTUI

UNIVERSITY OF INDONESIA

2026

Project Information

Project Title : Network Intrusion Detection Using Zeek with Grafana Visualization and Discord Alerting

Batch Code : 4CS1

Start Date : 5 March 2026

End Date : 2 April 2026

Name of Faculty : Tri Agus Riyadi

Names of Developers :

1. Muhammad Farrel Rachmadya Pasya
2. Reyandanu Fajri Daniswara

Date of Submission: 2 April 2026S

Acknowledgement

First of all, thanks to Allah SWT, who has given us the opportunity to complete this task. Blessing and salutation may also be upon the most honorable prophet Muhammad SAW who has given us life guidance from our birth to the end of life. We also want to express our thanks to Mr. Tri Agus Riyadi, S.Kom., M.T. as our lecturer and consultant, who has helped us to direct the preparation of paper into reality. Without help from the lecturer, this work would not have been possible.

Depok, 3rd February 2026

Author

System Analysis

This document reports on a simulation conducted across three Virtual Machines. One acts as the attacker, one acts as the client being monitored by Zeek, and the third runs the dashboard that receives logs from the client.

The project demonstrates how the dashboard visualises network attacks in real time. The attacker targets the client, while Zeek captures network traffic on the client machine and forwards the logs to the dashboard server using a custom Python script. A separate blocking script detects the attacker's IP address, which can then be submitted through a dedicated panel in the Grafana dashboard to block it at the network level.

Environment Setup

Host Machine: Arch Linux (Lenovo Legion Slim 5i)

Virtualization: QEMU/KVM via Virt-Manager

Network: libvirt default NAT Network (192.168.122.0/24)

VM	OS	IP	Role
ubuntu-zeek	Ubuntu Server 22.04	192.168.122.108	Victim + Zeek
kali-attacker	Kali Linux	192.168.122.122	Attacker
Arch host	Arch Linux	192.168.122.1	Loki + Grafana

Configuration

- Loki and Grafana Installation in Arch Host:

```
docker run -d --name loki --network host -v ~/zeek-project/loki-  
config.yaml:/etc/loki/local-config.yaml grafana/loki:latest -  
config.file=/etc/loki/local-config.yaml
```

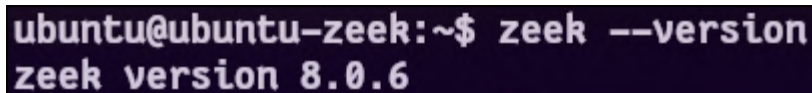
```
docker run -d --name grafana --network host -e  
"GF_PANELS_DISABLE_SANITIZE_HTML=true" grafana/grafana:latest
```



CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
145af2570f9b	grafana/grafana:latest	"/run.sh"	7 days ago	Up 15 minutes		grafana
0aff2311d672	grafana/loki:latest	"/usr/bin/loki -conf..."	10 days ago	Up 15 minutes		loki

- Zeek Installation in Ubuntu Server 22.04 VM:

```
echo 'deb  
http://download.opensuse.org/repositories/security:/zeek/xUbuntu_22.04/ /'  
| sudo tee /etc/apt/sources.list.d/zeek.list  
  
curl -fsSL  
https://download.opensuse.org/repositories/security:zeek/xUbuntu_22.04/Rele  
ase.key | sudo gpg --dearmor | sudo tee /etc/apt/trusted.gpg.d/zeek.gpg >  
/dev/null  
  
sudo apt update  
  
sudo apt install zeek -y
```



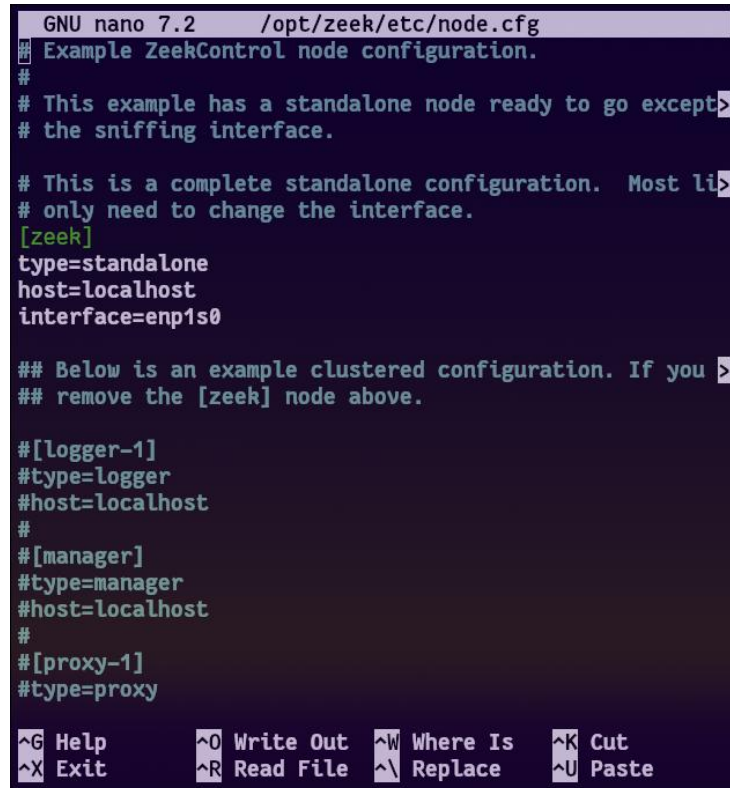
```
ubuntu@ubuntu-zeek:~$ zeek --version  
zeek version 8.0.6
```

Configuration

Add this in node.cfg on Ubuntu Server VM:

```
sudo nano /opt/zeek/etc/node.cfg
```

```
[zeek]
type=standalone
host=localhost
interface=enp1s0
```



```
GNU nano 7.2 /opt/zeek/etc/node.cfg
## Example ZeekControl node configuration.
#
# This example has a standalone node ready to go except
# the sniffing interface.
#
# This is a complete standalone configuration. Most li
# only need to change the interface.
[zeek]
type=standalone
host=localhost
interface=enp1s0

## Below is an example clustered configuration. If you
## remove the [zeek] node above.

#[logger-1]
#type=logger
#host=localhost
#
#[manager]
#type=manager
#host=localhost
#
#[proxy-1]
#type=proxy

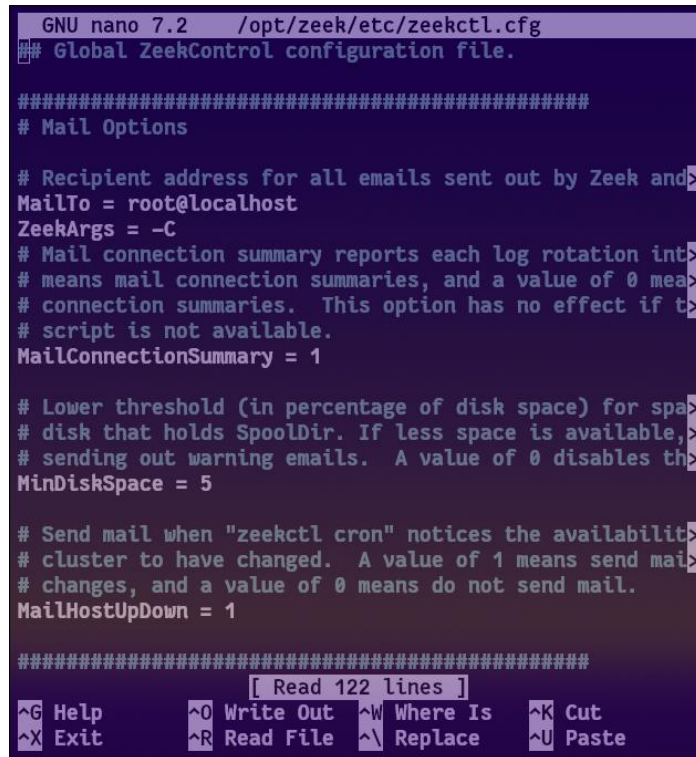
^G Help      ^O Write Out  ^W Where Is   ^K Cut
^X Exit      ^R Read File  ^\ Replace    ^U Paste
```

Configuration

Add this in zeekctl.cfg on Ubuntu Server VM:

```
sudo nano /opt/zeek/etc/zeekctl.cfg
```

```
ZeekArgs = -C
```



```
GNU nano 7.2 /opt/zeek/etc/zeekctl.cfg
## Global ZeekControl configuration file.

#####
# Mail Options

# Recipient address for all emails sent out by Zeek and
MailTo = root@localhost
ZeekArgs = -C
# Mail connection summary reports each log rotation into
# means mail connection summaries, and a value of 0 means
# connection summaries. This option has no effect if the
# script is not available.
MailConnectionSummary = 1

# Lower threshold (in percentage of disk space) for space
# disk that holds SpoolDir. If less space is available,
# sending out warning emails. A value of 0 disables this.
MinDiskSpace = 5

# Send mail when "zeekctl cron" notices the availability
# cluster to have changed. A value of 1 means send mail
# changes, and a value of 0 means do not send mail.
MailHostUpDown = 1

#####
[ Read 122 lines ]
^G Help      ^O Write Out  ^W Where Is   ^K Cut
^X Exit      ^R Read File  ^\ Replace    ^U Paste
```


Configuration

Make this custom detection script on Ubuntu Server VM:

```
sudo nano /opt/zeek/share/zeek/site/detect-scan.zeek
```

```
GNU nano 7.2
# Custom port scan detection for Zeek 8
module PortScan;

export {
  redef enum Notice::Type += {
    Port_Scan
  };
}

global distinct_ports: table[addr] of set[port] &create_expire=60sec;

event new_connection(c: connection)
{
  local src = c$id$orig_h;
  local dst_port = c$id$resp_p;

  if ( src !in distinct_ports )
    distinct_ports[src] = set();

  add distinct_ports[src][dst_port];

  if ( |distinct_ports[src]| > 20 )
  {
    NOTICE([$note=Port_Scan,
             $msg=fmt("Port scan detected from %s (%d ports)",
                      src, |distinct_ports[src]|),
             $src=src,
             $identifier=cat(src)]);
  }
}
```

Configuration

Add local policy to local.zeeek on Ubuntu Server VM:

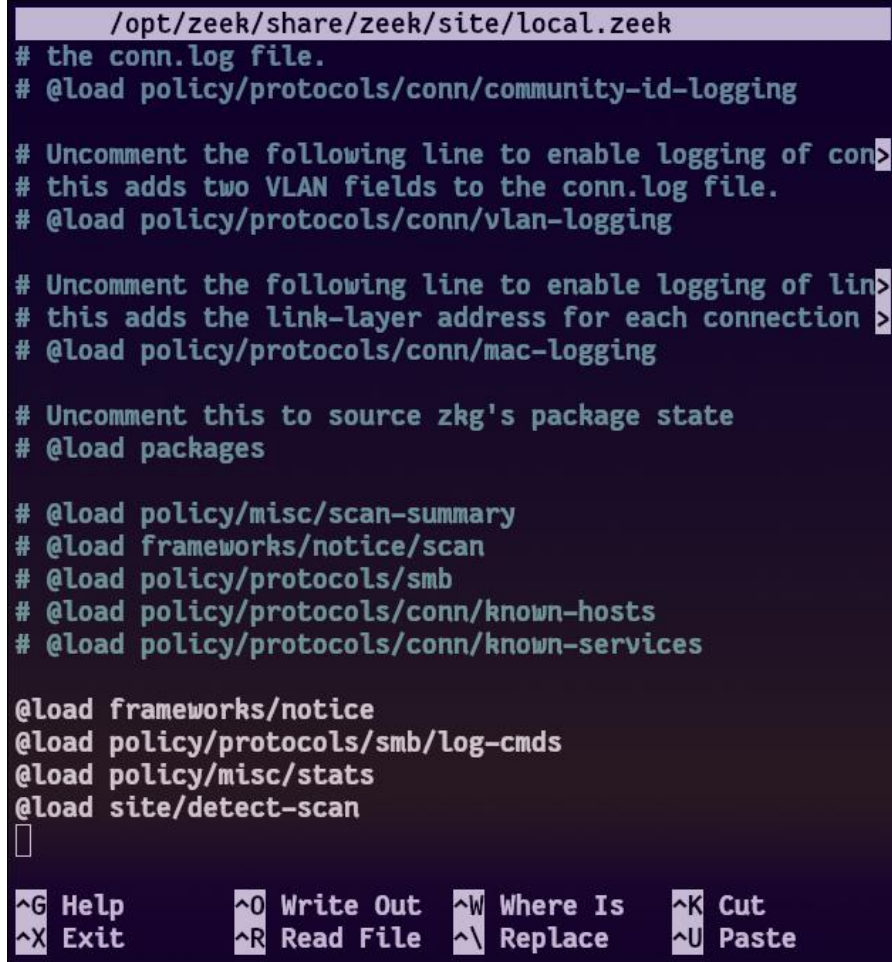
```
sudo nano /opt/zeek/share/zeek/site/local.zeeek
```

```
@load frameworks/notice
```

```
@load policy/misc/stats
```

```
@load policy/protocols/smb/log-cmds
```

```
@load site/detect-scan
```



```
/opt/zeek/share/zeek/site/local.zeeek
# the conn.log file.
# @load policy/protocols/conn/community-id-logging

# Uncomment the following line to enable logging of conn>
# this adds two VLAN fields to the conn.log file.
# @load policy/protocols/conn/vlan-logging

# Uncomment the following line to enable logging of lin>
# this adds the link-layer address for each connection >
# @load policy/protocols/conn/mac-logging

# Uncomment this to source zkg's package state
# @load packages

# @load policy/misc/scan-summary
# @load frameworks/notice/scan
# @load policy/protocols/smb
# @load policy/protocols/conn/known-hosts
# @load policy/protocols/conn/known-services

@load frameworks/notice
@load policy/protocols/smb/log-cmds
@load policy/misc/stats
@load site/detect-scan

```

Help **Write Out** **Where Is** **Cut**
Exit **Read File** **Replace** **Paste**

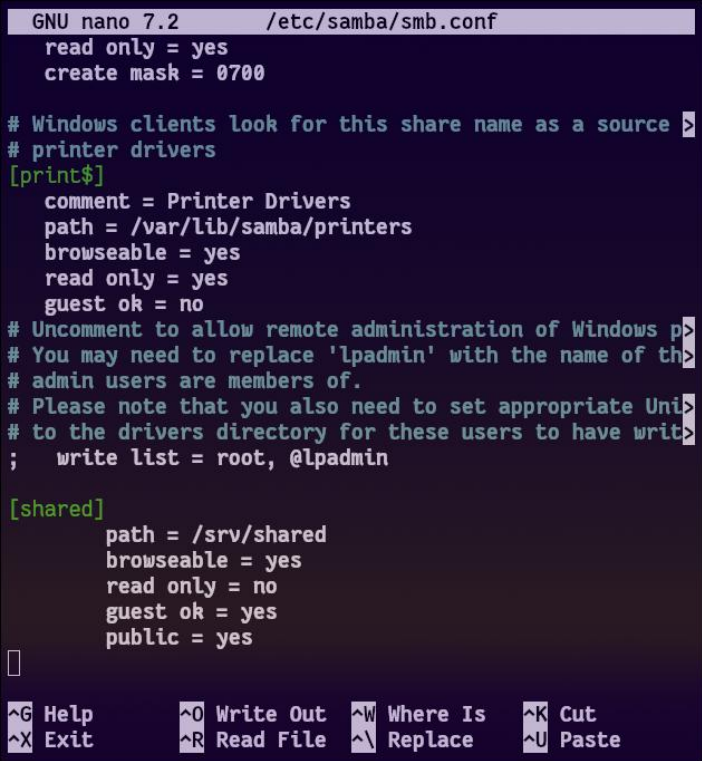
Configuration

SMB Configuration in Ubuntu Server VM:

```
sudo mkdir -p /srv/shared
sudo chmod 777 /srv/shared
echo "confidential data" | sudo tee /srv/shared/secret.txt
sudo nano /etc/samba/smb.conf
```

in smb.conf add this:

```
[shared]
path = /srv/shared
browseable = yes
read only = no
guest ok = yes
public = yes
```



```
GNU nano 7.2 /etc/samba/smb.conf
read only = yes
create mask = 0700

# Windows clients look for this share name as a source >
# printer drivers
[print$]
comment = Printer Drivers
path = /var/lib/samba/printers
browseable = yes
read only = yes
guest ok = no
# Uncomment to allow remote administration of Windows p>
# You may need to replace 'lpadmin' with the name of th>
# admin users are members of.
# Please note that you also need to set appropriate Uni>
# to the drivers directory for these users to have writ>
; write list = root, @lpadmin

[shared]
path = /srv/shared
browseable = yes
read only = no
guest ok = yes
public = yes

^G Help      ^O Write Out ^W Where Is  ^K Cut
^X Exit      ^R Read File ^\ Replace  ^U Paste
```

Configuration

Deploy Zeek on Ubuntu Server VM:

```
sudo /opt/zeek/bin/zeekctl deploy
```

```
ubuntu@ubuntu-zeek:~$ sudo nano /etc/samba/smb.conf
[sudo] password for ubuntu:
Error opening terminal: xterm-kitty.
<udo /opt/zeek/bin/zeekctl deploy
checking configurations ...
installing ...
removing old policies in /opt/zeek/spool/installed-scrip
ts-do-not-touch/site ...
removing old policies in /opt/zeek/spool/installed-scrip
ts-do-not-touch/auto ...
creating policy directories ...
installing site policies ...
generating standalone-layout.zeek ...
generating local-networks.zeek ...
generating zeekctl-config.zeek ...
generating zeekctl-config.sh ...
stopping ...
stopping zeek ...
starting ...
starting zeek ...
ubuntu@ubuntu-zeek:~$
```

```
sudo /opt/zeek/bin/zeekctl status
```

```
ubuntu@ubuntu-zeek:~$ sudo /opt/zeek/bin/zeekctl status
```

Name	Type	Host	Status	Pid	Started
zeek	standalone	localhost	running	4026	24 Mar 13:10:17

Python Script

- **zeek_forwarder.py**

The forwarder monitors five Zeek log files simultaneously using threading, forwards all entries to Loki, and sends Discord alerts for security-relevant log types

- **block_agent.py**

The block agent is a Flask REST API that receives commands from the Grafana dashboard and executes iptables rules on the Ubuntu VM.

After these scripts is made run

```
python3 zeek_forwarder.py  
python3 block_agent.py
```

The content for both of this script will be pasted below:

Python Script (zeek_forwarder.py)

```
#!/usr/bin/env python3

import time
import requests
import threading

# — Configuration —————
LOKI_URL = "http://192.168.122.1:3100/loki/api/v1/push"
DISCORD_WEBHOOK_URL = " discord webhook url "
GRAFANA_URL = "http://192.168.122.1:3000"
BLOCK_AGENT_URL = "http://localhost:5000"

LOG_FILES = {
    "notice":      "/opt/zeek/spool/zeek/notice.log",
    "conn":        "/opt/zeek/spool/zeek/conn.log",
    "smb_mapping": "/opt/zeek/spool/zeek/smb_mapping.log",
    "smb_files":   "/opt/zeek/spool/zeek/smb_files.log",
    "smb_cmd":     "/opt/zeek/spool/zeek/smb_cmd.log",
}

ALERT_LOG_TYPES = ["notice", "smb_mapping", "smb_files", "smb_cmd"]
```

Python Script (zeek_forwarder.py)

```
DISCORD_COOLDOWN = 30
last_discord_time = {}

# ——— Loki —————
def push_to_loki(line, log_type):
    ts = str(int(time.time() * 1e9))
    payload = {
        "streams": [{
            "stream": {"job": "zeek", "log_type": log_type},
            "values": [[ts, line.strip()]]
        }]
    }
    try:
        resp = requests.post(LOKI_URL, json=payload, timeout=5)
        if resp.status_code not in (200, 204):
            print(f"[WARN] Loki returned {resp.status_code}")
    except Exception as e:
        print(f"[ERROR] Loki push failed: {e}")

# ——— Block Agent —————
def report_attacker_to_agent(ip):
```

Python Script (zeek_forwarder.py)

```
"""Send latest attacker IP to block agent"""

try:
    requests.post(
        f"{BLOCK_AGENT_URL}/report_attacker",
        json={"ip": ip},
        timeout=3
    )
    print(f"[AGENT] Reported attacker: {ip}")
except Exception as e:
    print(f"[WARN] Could not report to block agent: {e}")

# ——— Discord —————
def extract_ip(line, log_type):
    try:
        parts = line.strip().split('\t')
        if log_type == "notice" and len(parts) > 13:
            return parts[13]
        elif log_type in ("smb_mapping", "smb_files", "smb_cmd") and
len(parts) > 2:
            return parts[2]
        return None
    except Exception:
        return None
```


Python Script (zeek_forwarder.py)

```
def get_alert_title(log_type):
    titles = {
        "notice":      "🚨 Port Scan Detected",
        "smb_mapping": "⚠️ SMB Share Enumeration Detected",
        "smb_files":   "🔴 SMB Unauthorized File Access",
        "smb_cmd":     "🔴 SMB Command Executed",
    }
    return titles.get(log_type, "⚠️ Zeek Alert")

def get_alert_color(log_type):
    colors = {
        "notice":      0xFF0000,
        "smb_mapping": 0xFF8C00,
        "smb_files":   0xFF0000,
        "smb_cmd":     0xFF4500,
    }
    return colors.get(log_type, 0xFF0000)

def send_discord_alert(line, log_type):
    src_ip = extract_ip(line, log_type)

    description = (
        f"***Time:** {time.strftime('%Y-%m-%d %H:%M:%S')}\n"
```

Python Script (zeek_forwarder.py)

```
f"***Type:** {log_type.upper()}\n"
)
if src_ip and src_ip != '-':
    description += f"***Source IP:** `{src_ip}`\n"

description += f"\n***Raw Log:**\n```${line.strip()[:300]}```"

payload = {
    "embeds": [{
        "title": get_alert_title(log_type),
        "description": description,
        "color": get_alert_color(log_type),
        "footer": {
            "text": f"Zeek IDS • {time.strftime('%Y-%m-%d %H:%M:%S')}"
        }
    }],
    "components": [{
        "type": 1,
        "components": [{
            "type": 2,
            "style": 5,
            "label": "🔗 Open Grafana Dashboard",
            "url": GRAFANA_URL
        ]
    }]
```

Python Script (zeek_forwarder.py)

```
        ]]  
    ]]  
}  
try:  
    requests.post(DISCORD_WEBHOOK_URL, json=startup_payload, timeout=5)  
except Exception as e:  
    print(f"[ERROR] Startup Discord message failed: {e}")  
  
threads = []  
for log_type, filepath in LOG_FILES.items():  
    t = threading.Thread(  
        target=watch,  
        args=(log_type, filepath),  
        daemon=True  
    )  
    t.start()  
    threads.append(t)  
  
print("[*] All watchers running. Press Ctrl+C to stop.\n")  
  
try:  
    while True:  
        time.sleep(1)
```

Python Script (zeek_forwarder.py)

```
except KeyboardInterrupt:

    shutdown_payload = {

        "embeds": [{

            "title": "🔴 Zeek IDS Stopped",

            "description": f"**Time:** {time.strftime('%Y-%m-%d\n%H:%M:%S')}",

            "color": 0xFF0000

        }]

    }

    try:

        requests.post(DISCORD_WEBHOOK_URL, json=shutdown_payload,
timeout=5)

    except:

        pass

    print("\n[*] Stopped.")

if __name__ == "__main__":

    main()
```

Python Script (Block_Agent.py)

```
#!/usr/bin/env python3
from flask import Flask, request, jsonify
from flask_cors import CORS
import subprocess
import time
import requests

app = Flask(__name__)
CORS(app)

DISCORD_WEBHOOK_URL = "discord webhook url"

# Track blocked IPs and latest attacker
blocked_ips = {}
latest_attacker = None

def send_discord(message, color=0xFF0000):
    payload = {
        "embeds": [{
            "title": "🔴 IP Block Action",
            "description": message,
            "color": color,
            "footer": {
```

Python Script (Block_Agent.py)

```
        "text": f"Zeek IDS • {time.strftime('%Y-%m-%d %H:%M:%S')}"
    }
    ]]
}
try:
    requests.post(DISCORD_WEBHOOK_URL, json=payload, timeout=5)
except Exception as e:
    print(f"[ERROR] Discord: {e}")

def block_ip(ip):
    try:
        check = subprocess.run(
            ["sudo", "iptables", "-C", "INPUT", "-s", ip, "-j", "DROP"],
            capture_output=True
        )
        if check.returncode == 0:
            return False, f"{ip} is already blocked"
        subprocess.run(
            ["sudo", "iptables", "-I", "INPUT", "-s", ip, "-j", "DROP"],
            check=True
        )
        blocked_ips[ip] = time.strftime('%Y-%m-%d %H:%M:%S')
        return True, f"{ip} blocked successfully"
```

Python Script (Block_Agent.py)

```
except Exception as e:
    return False, str(e)

def unblock_ip(ip):
    try:
        subprocess.run(
            ["sudo", "iptables", "-D", "INPUT", "-s", ip, "-j", "DROP"],
            check=True
        )
        if ip in blocked_ips:
            del blocked_ips[ip]
        return True, f"{ip} unblocked successfully"
    except Exception as e:
        return False, str(e)

@app.route('/block', methods=['POST'])
def handle_block():
    data = request.json
    ip = data.get('ip')
    if not ip:
        return jsonify({"status": "error", "message": "No IP provided"}),
400
    success, message = block_ip(ip)
    if success:
```

Python Script (Block_Agent.py)

```
send_discord(
    f"✅ **IP Unblocked**\n"
    f"-----\n"
    f"***IP:** `{ip}`\n"
    f"***Time:** {time.strftime('%Y-%m-%d %H:%M:%S')}\n"
    f"***Action:** Removed from iptables",
    color=0x00FF00
)
print(f"[UNBLOCKED] {ip}")
return jsonify({"status": "success", "message": message}), 200
else:
    return jsonify({"status": "error", "message": message}), 400

@app.route('/blocked', methods=['GET'])
def list_blocked():
    return jsonify({"status": "success", "blocked_ips": blocked_ips}), 200

@app.route('/latest_attacker', methods=['GET'])
def get_latest_attacker():
    return jsonify({"status": "success", "ip": latest_attacker}), 200

@app.route('/report_attacker', methods=['POST'])
def report_attacker():
```


Python Script (Block_Agent.py)

```
global latest_attacker

data = request.json

ip = data.get('ip')

if ip and ip not in ('127.0.0.1', '192.168.122.1', '-'):

    latest_attacker = ip

    print(f"[ATTACKER] Latest attacker updated: {ip}")

    return jsonify({"status": "success"}), 200

return jsonify({"status": "ignored"}), 200


@app.route('/health', methods=['GET'])
def health():

    return jsonify({"status": "ok"}), 200


if __name__ == "__main__":

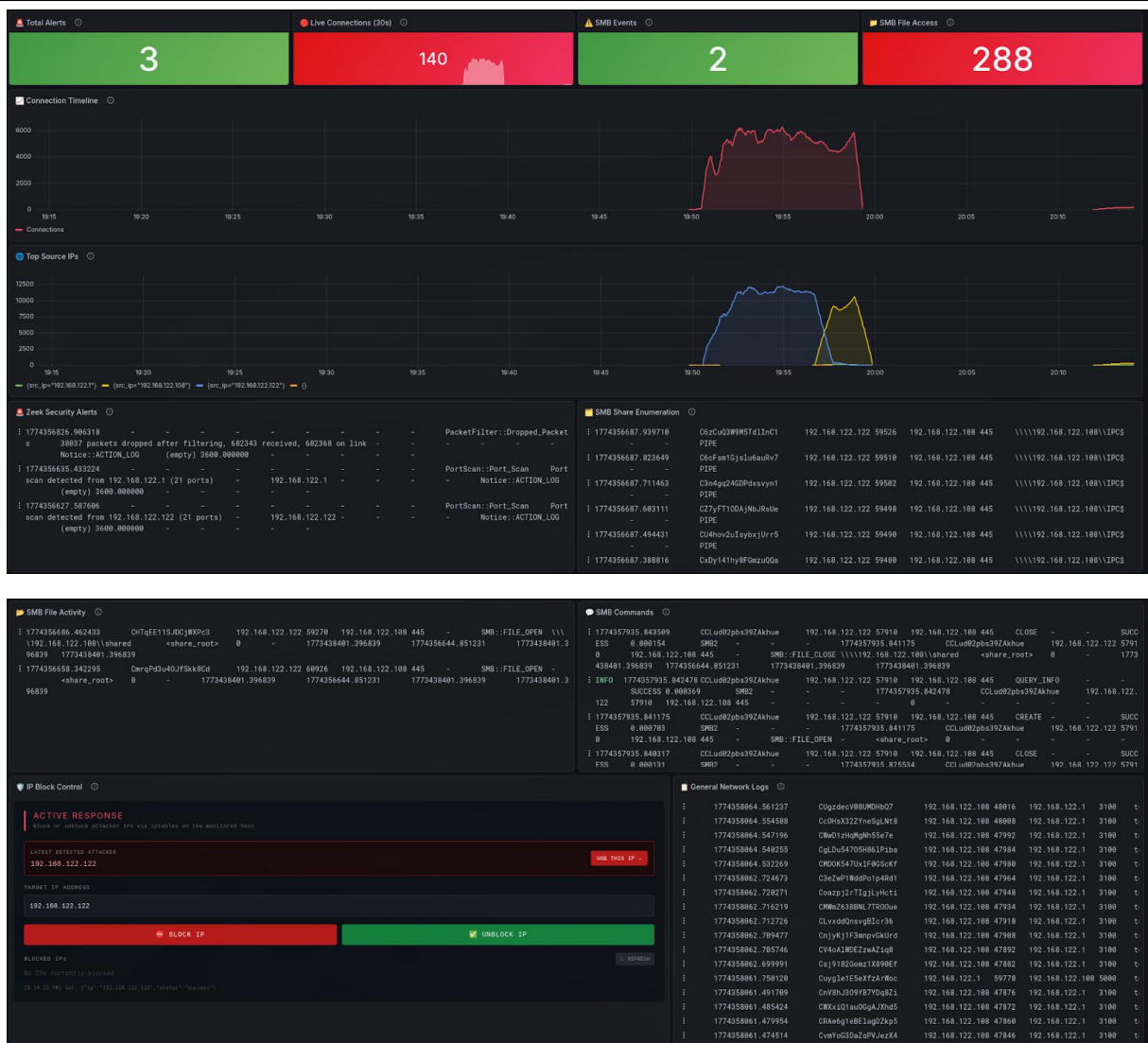
    print("=" * 50)

    print("  Zeek Block Agent running on port 5000")

    print("=" * 50)

    app.run(host='0.0.0.0', port=5000, debug=False)
```

Grafana Dashboard Preview



The json file below is the configuration file for this exact dashboard.

Grafana Dashboard Json Configuration

```
{
  "__inputs": [
    {
      "name": "DS_LOKI",
      "label": "Loki",
      "type": "datasource",
      "pluginId": "loki",
      "pluginName": "Loki"
    }
  ],
  "title": "Zeek IDS - Network Intrusion Detection",
  "uid": "zeek-ids-v2",
  "schemaVersion": 39,
  "version": 1,
  "refresh": "5s",
  "time": { "from": "now-1h", "to": "now" },
  "panels": [
    {
      "id": 1,
      "type": "stat",
      "title": "🚨 Total Alerts",
      "description": "Total number of Zeek security alerts generated in the selected time range. Includes port scans, SMB enumeration, and unauthorized access attempts.",
    }
  ]
}
```

Grafana Dashboard Json Configuration

```
"gridPos": { "x": 0, "y": 0, "w": 6, "h": 4 },
"datasource": { "type": "loki", "uid": "${DS_LOKI}" },
"targets": [{
  "expr": "count_over_time({job=\"zeek\", log_type=\"notice\"}
[3h])",
  "refId": "A"
}],
"options": {
  "reduceOptions": { "calcs": ["lastNotNull"] },
  "colorMode": "background",
  "graphMode": "none",
  "textMode": "auto"
},
"fieldConfig": {
  "defaults": {
    "thresholds": {
      "mode": "absolute",
      "steps": [
        { "color": "green", "value": 0 },
        { "color": "yellow", "value": 10 },
        { "color": "red", "value": 50 }
      ]
    }
  }
}
```

Grafana Dashboard Json Configuration

```
}  
  
,  
  
{  
  "id": 2,  
  "type": "stat",  
  "title": "● Live Connections (30s)",  
  "description": "Number of active network connections observed by Zeek  
in the last 30 seconds. A sudden spike may indicate a port scan or flood  
attack.",  
  "gridPos": { "x": 6, "y": 0, "w": 6, "h": 4 },  
  "datasource": { "type": "loki", "uid": "${DS_LOKI}" },  
  "targets": [{  
    "expr": "count_over_time({job=\"zeek\", log_type=\"conn\"} [30s])",  
    "refId": "A"  
  }],  
  "options": {  
    "reduceOptions": { "calcs": ["lastNotNull"] },  
    "colorMode": "background",  
    "graphMode": "area"  
  },  
  "fieldConfig": {  
    "defaults": {  
      "thresholds": {  
        "mode": "absolute",
```

Grafana Dashboard Json Configuration

```
      "steps": [
        { "color": "green", "value": 0 },
        { "color": "yellow", "value": 50 },
        { "color": "red", "value": 100 }
      ]
    }
  }
},
{
  "id": 3,
  "type": "stat",
  "title": "⚠️ SMB Events",
  "description": "Total SMB (Server Message Block) events detected. SMB
is commonly targeted for lateral movement, credential theft, and
unauthorized file access.",
  "gridPos": { "x": 12, "y": 0, "w": 6, "h": 4 },
  "datasource": { "type": "loki", "uid": "${DS_LOKI}" },
  "targets": [{
    "expr": "count_over_time({job=\"zeek\", log_type=\"smb_mapping\"}
[1h])",
    "refId": "A"
  }],
  "options": {
    "reduceOptions": { "calcs": ["lastNotNull"] },
```

Grafana Dashboard Json Configuration

```
    "colorMode": "background",
    "graphMode": "none"
  },
  "fieldConfig": {
    "defaults": {
      "thresholds": {
        "mode": "absolute",
        "steps": [
          { "color": "green", "value": 0 },
          { "color": "orange", "value": 5 },
          { "color": "red", "value": 20 }
        ]
      }
    }
  },
  {
    "id": 4,
    "type": "stat",
    "title": "📁 SMB File Access",
    "description": "Number of file access attempts detected on SMB shares. Unauthorized file reads or writes indicate an attacker actively exfiltrating or tampering with data.",
    "gridPos": { "x": 18, "y": 0, "w": 6, "h": 4 },
```

Grafana Dashboard Json Configuration

```
"datasource": { "type": "loki", "uid": "${DS_LOKI}" },
"targets": [{
  "expr": "count_over_time({job=\"zeek\", log_type=\"smb_files\"}
[1h])",
  "refId": "A"
}],
"options": {
  "reduceOptions": { "calcs": ["lastNotNull"] },
  "colorMode": "background",
  "graphMode": "none"
},
"fieldConfig": {
  "defaults": {
    "thresholds": {
      "mode": "absolute",
      "steps": [
        { "color": "green", "value": 0 },
        { "color": "orange", "value": 3 },
        { "color": "red", "value": 10 }
      ]
    }
  }
}
```


Grafana Dashboard Json Configuration

```
{
  "id": 5,
  "type": "timeseries",
  "title": "📊 Connection Timeline",
  "description": "Real-time graph of all network connections observed by Zeek over time. A sharp spike indicates reconnaissance activity such as an Nmap port scan. Normal baseline should be relatively flat.",
  "gridPos": { "x": 0, "y": 4, "w": 24, "h": 8 },
  "datasource": { "type": "loki", "uid": "${DS_LOKI}" },
  "targets": [{
    "expr": "count_over_time({job=\"zeek\", log_type=\"conn\"} [30s])",
    "refId": "A",
    "legendFormat": "Connections"
  }],
  "options": {
    "legend": { "displayMode": "list", "placement": "bottom" },
    "tooltip": { "mode": "single" }
  },
  "fieldConfig": {
    "defaults": {
      "custom": {
        "lineWidth": 2,
        "fillOpacity": 15,
        "drawStyle": "line",

```

Grafana Dashboard Json Configuration

```
        "lineInterpolation": "smooth"
      },
      "color": { "mode": "fixed", "fixedColor": "#F2495C" },
      "thresholds": {
        "mode": "absolute",
        "steps": [
          { "color": "green", "value": 0 },
          { "color": "red", "value": 100 }
        ]
      }
    }
  },
  {
    "id": 6,
    "type": "timeseries",
    "title": "🌐 Top Source IPs",
    "description": "Tracks the top 10 source IP addresses by connection count over time. During an attack the attacker IP will appear as a dominant spike. Use this panel to identify which IP to block.",
    "gridPos": { "x": 0, "y": 12, "w": 24, "h": 8 },
    "datasource": { "type": "loki", "uid": "${DS_LOKI}" },
    "targets": [{
      "expr": "topk(10,\n  sum by (src_ip)
(\n    count_over_time(\n      {job=\"zeek\", log_type=\"conn\"}\n      |
regex `(?P<src_ip>\\d+\\.\\.\\d+\\.\\.\\d+\\.\\.\\d+)`\\n      [1m]\\n    )\\n  )\\n)",
```

Grafana Dashboard Json Configuration

```
    "refId": "A"
  }],
  "options": {
    "legend": { "displayMode": "list", "placement": "bottom" },
    "tooltip": { "mode": "single" }
  },
  "fieldConfig": {
    "defaults": {
      "custom": {
        "lineWidth": 2,
        "fillOpacity": 10,
        "drawStyle": "line",
        "lineInterpolation": "smooth"
      }
    }
  }
},
{
  "id": 7,
  "type": "logs",
  "title": "🚨 Zeek Security Alerts",
  "description": "Live feed of all security notices generated by Zeek. This includes port scan detections (PortScan::Port_Scan), dropped packets, and any custom detection rules. Each entry shows the alert type, source IP, and details.",
```

Grafana Dashboard Json Configuration

```
"gridPos": { "x": 0, "y": 20, "w": 12, "h": 9 },
"datasource": { "type": "loki", "uid": "${DS_LOKI}" },
"targets": [{
  "expr": "{job=\"zeek\", log_type=\"notice\"}",
  "refId": "A"
}],
"options": {
  "dedupStrategy": "none",
  "enableLogDetails": true,
  "sortOrder": "Descending",
  "wrapLogMessage": true
}
},
{
  "id": 8,
  "type": "logs",
  "title": "🚩 SMB Share Enumeration",
  "description": "Logs of SMB share mapping activity. When an attacker runs tools like enum4linux or smbclient to list available shares, it appears here. Shows source IP, share name, and access time.",
  "gridPos": { "x": 12, "y": 20, "w": 12, "h": 9 },
  "datasource": { "type": "loki", "uid": "${DS_LOKI}" },
  "targets": [{
    "expr": "{job=\"zeek\", log_type=\"smb_mapping\"}",
```

Grafana Dashboard Json Configuration

```
      "refId": "A"
    }],
    "options": {
      "dedupStrategy": "none",
      "enableLogDetails": true,
      "sortOrder": "Descending",
      "wrapLogMessage": true
    }
  },
  {
    "id": 9,
    "type": "logs",
    "title": "📁 SMB File Activity",
    "description": "Records of file operations performed over SMB. Shows when an attacker accesses, reads, or downloads files from shared directories. Critical for detecting data exfiltration.",
    "gridPos": { "x": 0, "y": 29, "w": 12, "h": 9 },
    "datasource": { "type": "loki", "uid": "${DS_LOKI}" },
    "targets": [{
      "expr": "{job=\"zeek\", log_type=\"smb_files\"}",
      "refId": "A"
    }],
    "options": {
      "dedupStrategy": "none",
```

Grafana Dashboard Json Configuration

```
    "enableLogDetails": true,
    "sortOrder": "Descending",
    "wrapLogMessage": true
  },
  {
    "id": 10,
    "type": "logs",
    "title": "🗨 SMB Commands",
    "description": "Detailed log of SMB commands executed during a session. Shows the exact commands sent by the attacker such as directory listings, file reads, and writes. Useful for forensic reconstruction of the attack.",
    "gridPos": { "x": 12, "y": 29, "w": 12, "h": 9 },
    "datasource": { "type": "loki", "uid": "${DS_LOKI}" },
    "targets": [{
      "expr": "{job=\"zeek\", log_type=\"smb_cmd\"}",
      "refId": "A"
    }],
    "options": {
      "dedupStrategy": "none",
      "enableLogDetails": true,
      "sortOrder": "Descending",
      "wrapLogMessage": true
    }
  }
```

Grafana Dashboard Json Configuration

```
},  
{  
  "id": 11,  
  "type": "text",  
  "title": " 🟢 IP Block Control",  
  "description": "Active response panel. Use this to block or unblock attacker IPs directly from the dashboard using iptables on the monitored Ubuntu VM. Check the Top Source IPs panel above to identify the attacker IP.",  
  "gridPos": { "x": 0, "y": 38, "w": 14, "h": 14 },  
  "options": {  
    "mode": "html",  
    "content": "<div style='padding:20px;font-family:'Courier New',monospace;background:#0d1117;border-radius:8px;'>\n  <div style='border-left:3px solid #F2495C;padding-left:15px;margin-bottom:20px;'>\n    <h3 style='color:#F2495C;margin:0 0 4px 0;font-size:16px;letter-spacing:2px;'>ACTIVE RESPONSE</h3>\n    <p style='color:#666;margin:0;font-size:11px;'>Block or unblock attacker IPs via iptables on the monitored host</p>\n  </div>\n\n  <div style='margin-bottom:15px;'>\n    <label style='color:#888;font-size:11px;letter-spacing:1px;display:block;margin-bottom:6px;'>TARGET IP ADDRESS</label>\n    <input id='ipInput' type='text' placeholder='e.g. 192.168.122.122'\n      style='width:100%;padding:10px;background:#161b22;border:1px solid #30363d;border-radius:4px;color:#e6edf3;font-family:'Courier New';font-size:13px;box-sizing:border-box;' /\n    </div>\n\n  <div style='display:flex;gap:10px;margin-bottom:15px;'>\n    <button\n      onclick='blockIP()'\n      style='flex:1;padding:10px;background:#b91c1c;color:#fff;border:none;border-radius:4px;cursor:pointer;font-family:'Courier New';font-size:13px;letter-spacing:1px;'\n      onmouseover='this.style.background=#dc2626'\n      onmouseout='this.style.background=#b91c1c'>\n      🛑 BLOCK IP\n    </button>\n    <button\n      onclick='unblockIP()'\n      style='flex:1;padding:10px;background:#15803d;color:#fff;bor
```

Grafana Dashboard Json Configuration

```
der:none;border-radius:4px;cursor:pointer;font-family:'Courier New';font-
size:13px;letter-
spacing:1px;\n      onmouseover=\"this.style.background='#16a34a'\"
onmouseout=\"this.style.background='#15803d'\">\n       UNBLOCK
IP\n    </button>\n  </div>\n\n  <div style=\"margin-
bottom:15px;\n\">\n    <label style=\"color:#888;font-size:11px;letter-
spacing:1px;display:block;margin-bottom:6px;\n\">QUICK
BLOCK</label>\n    <button
onclick=\"quickBlock('192.168.122.122')\">\n      style=\"padding:6px
12px;background:#7f1d1d;color:#fca5a5;border:1px solid #991b1b;border-
radius:4px;cursor:pointer;font-family:'Courier New';font-
size:11px;\n      onmouseover=\"this.style.background='#991b1b'\"
onmouseout=\"this.style.background='#7f1d1d'\">\n       192.168.122.122
(Kali Attacker)\n    </button>\n  </div>\n\n  <div id=\"status\"
style=\"display:none;padding:8px 12px;border-radius:4px;font-
size:12px;margin-bottom:15px;\n\"></div>\n\n  <div>\n    <div
style=\"display:flex;justify-content:space-between;align-
items:center;margin-bottom:8px;\n\">\n      <label style=\"color:#888;font-
size:11px;letter-spacing:1px;\n\">BLOCKED IPs</label>\n      <button
onclick=\"loadBlocked()\">\n        style=\"padding:3px
8px;background:#21262d;color:#888;border:1px solid #30363d;border-
radius:3px;cursor:pointer;font-size:10px;font-family:'Courier
New';\n\">\n       REFRESH\n    </button>\n  </div>\n  <div
id=\"blockedList\" style=\"color:#666;font-size:12px;\n\">Click refresh to
load</div>\n</div>\n</div>\n\n<script>\nconst BLOCK_AGENT =
'http://192.168.122.108:5000';\n\nfunction getIP() { return
document.getElementById('ipInput').value.trim(); }\n\nfunction
setStatus(msg, ok) {\n  const el =
document.getElementById('status');\n  el.style.display =
'block';\n  el.style.background = ok ? '#14532d' :
'#7f1d1d';\n  el.style.color = ok ? '#86efac' :
'#fca5a5';\n  el.style.border = '1px solid ' + (ok ? '#166534' :
'#991b1b');\n  el.innerText = msg;\n  setTimeout(() => el.style.display =
'none', 4000);\n}\n\nfunction quickBlock(ip) {\n
document.getElementById('ipInput').value = ip; blockIP(); }\n\nasync
function blockIP() {\n  const ip = getIP();\n  if (!ip) { setStatus('⚠
Enter an IP address', false); return; }\n  try {\n    const r = await
fetch(BLOCK_AGENT+'/block', {method:'POST',headers:{'Content-
Type':'application/json'},body:JSON.stringify({ip})});\n    const d = await
r.json();\n    d.status==='success' ? setStatus('⊖ '+ip+' blocked', true)
: setStatus('X '+d.message, false);\n    loadBlocked();\n  } catch(e) {
```


Grafana Dashboard Json Configuration

```
Cannot reach block agent', false); }\n\n\nasync function unblockIP()
{\n  const ip = getIP();\n  if (!ip) { setStatus('⚠ Enter an IP address',
false); return; }\n  try {\n    const r = await
fetch(BLOCK_AGENT+'/unblock', {method:'POST',headers:{'Content-
Type':'application/json'},body:JSON.stringify({ip})});\n    const d = await
r.json();\n    d.status==='success' ? setStatus('✅ '+ip+' unblocked',
true) : setStatus('X '+d.message, false);\n    loadBlocked();\n  } catch(e)
{ setStatus('X Cannot reach block agent', false); }\n\n\nasync function
loadBlocked() {\n  try {\n    const r = await
fetch(BLOCK_AGENT+'/blocked');\n    const d = await r.json();\n    const
list = document.getElementById('blockedList');\n    if
(!Object.keys(d.blocked_ips).length) {\n      list.innerHTML = '<span
style=\"color:#444\">No IPs currently blocked</span>';\n    } else
{\n      list.innerHTML = Object.entries(d.blocked_ips).map(([ip,t])
=>\n        '<div style=\"display:flex;justify-content:space-between;align-
items:center;padding:5px 8px;background:#161b22;border:1px solid
#30363d;border-radius:3px;margin-bottom:4px;\">'+\n          '<span
style=\"color:#fca5a5;font-family:Courier New;font-size:11px;\">🛑\n
'+ip+'</span>'+\n          '<span style=\"color:#444;font-
size:10px;\">'+t+'</span>'+\n          '<button
onclick=\"quickUnblock(\\'+ip+'\\')\" style=\"padding:2px
8px;background:#15803d;color:#fff;border:none;border-
radius:3px;cursor:pointer;font-
size:10px;\">UNBLOCK</button>'+\n          '</div>'\n        ).join('');\n    }\n  } catch(e) { document.getElementById('blockedList').innerHTML='<span
style=\"color:#444\">Agent offline</span>'; }\n\n\nfunction
quickUnblock(ip) { document.getElementById('ipInput').value=ip;
unblockIP(); }\n\nloadBlocked();\nsetInterval(loadBlocked,
30000);\n</script>\"

  },

  {

    \"id\": 12,

    \"type\": \"logs\",

    \"title\": \"📄 General Network Logs\",

    \"description\": \"Full raw log stream from all Zeek log sources
combined. Shows everything Zeek is seeing on the network in real time. Useful for verifying
```

Grafana Dashboard Json Configuration

```
"gridPos": { "x": 14, "y": 38, "w": 10, "h": 14 },
"datasource": { "type": "loki", "uid": "${DS_LOKI}" },
"targets": [{
  "expr": "{job=\"zeek\"}",
  "refId": "A"
}],
"options": {
  "dedupStrategy": "none",
  "enableLogDetails": true,
  "sortOrder": "Descending",
  "wrapLogMessage": false
}
],
"templating": { "list": [] },
"annotations": { "list": [] },
"timepicker": {},
"timezone": "browser",
"tags": ["zeek", "ids", "security"]
}
```

Attack Demonstration

- Nmap port scanning demonstration

```
(kali@kali)-[~]
$ nmap -sC -sV -p- --min-rate=1000 192.168.122.108
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-24 20:15 +0700
```

- Grafana log alert:

1774358132.279797	-	-	-	-	-	-	-	PortScan::Port_Scan	Port
scan detected from 192.168.122.1 (21 ports)	-	-	-	192.168.122.1	-	-	-	Notice::ACTION_LOG	
(empty) 3600.000000	-	-	-	-	-	-	-		

- Discord alert:

 **Port Scan Detected**

Time: 2026-03-24 13:15:27

Type: NOTICE

Source IP: 192.168.122.122

Raw Log:

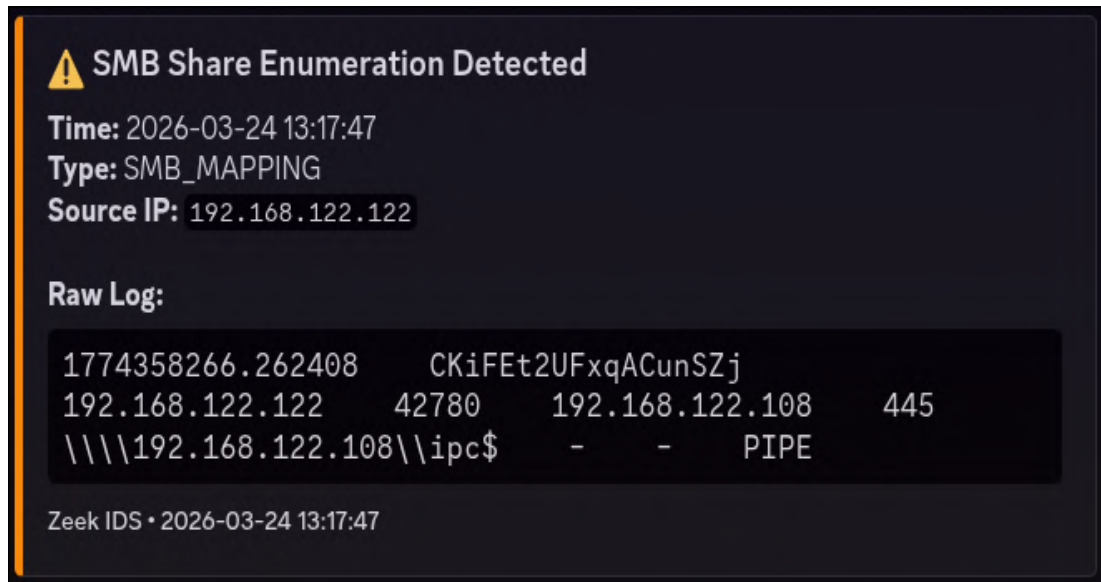
```
1774358124.655752 - - - - -
- - PortScan::Port_Scan Port scan detected from
192.168.122.122 (21 ports) - 192.168.122.122 -
- - Notice::ACTION_LOG (empty)
3600.000000 - - - - -
```

Zeek IDS • 2026-03-24 13:15:27

Attack Demonstration

- enum4 demonstration
enum4linux -a 192.168.122.108

- Discord alert:



Attack Demonstration

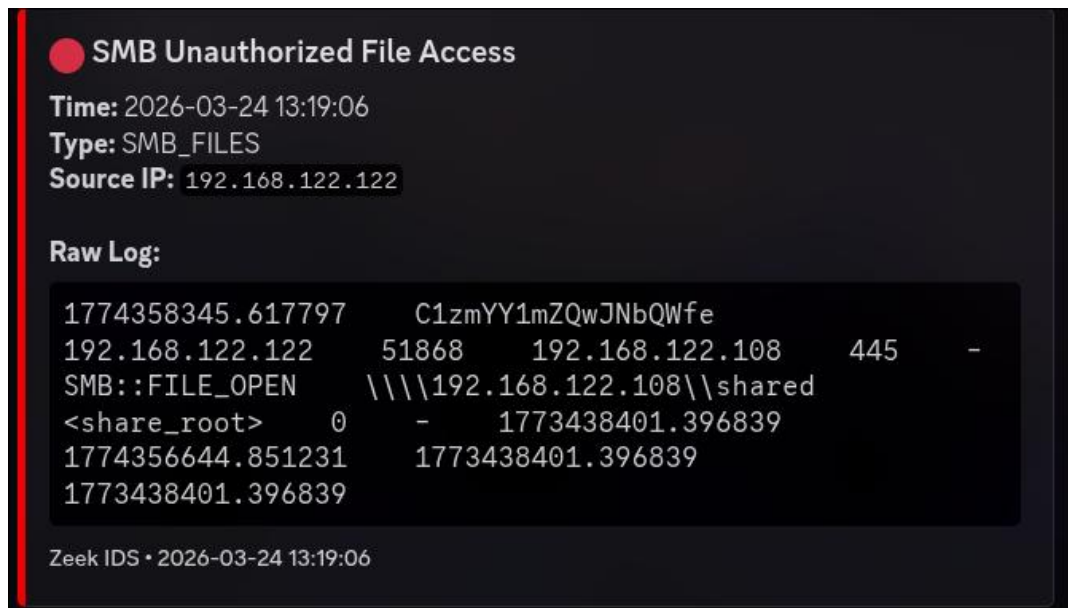
- Smb demonstration

```
smbclient //192.168.122.108/shared -N
```

```
smb: \> ls
```

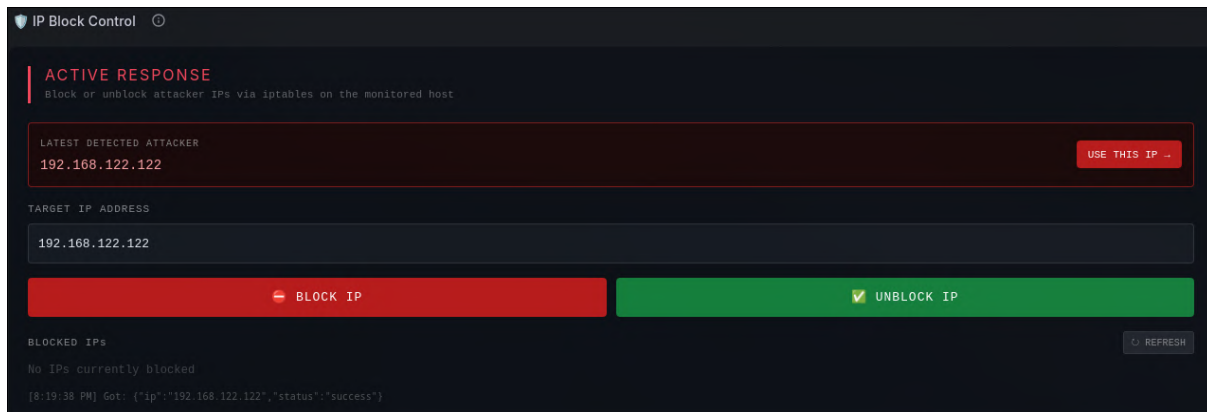
```
smb: \> get secret.txt
```

- Discord Alert:



Active Response

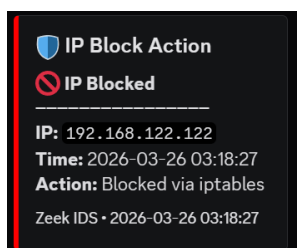
In grafana dashboard we have the blocking panel as shown below:



That dashboard shows after the attack which we can see there is latest detected attacker and we can press the button to use that ip to automatically input it into the target ip address section. And after we block the ip the script will use iptables to block the ip as it will show the blocked ips section as shown in the figure below:

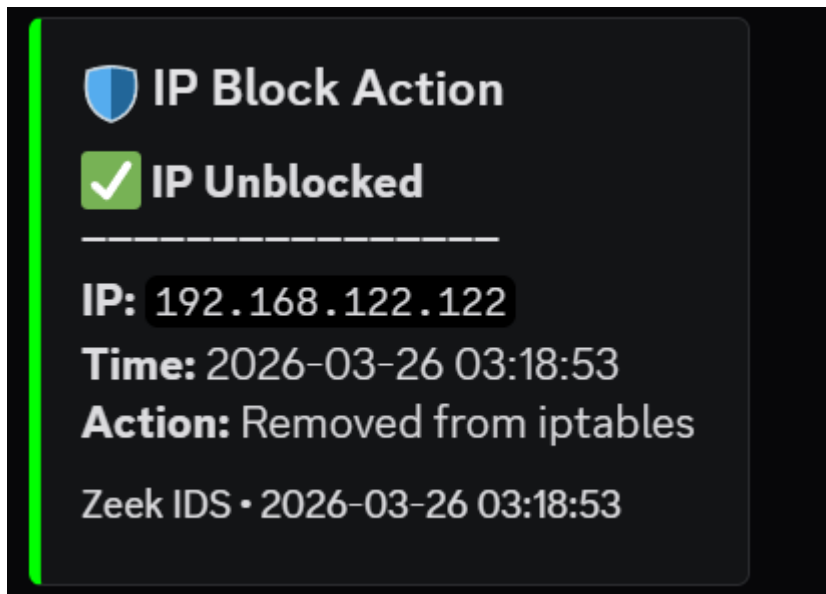


After the ip got blocked, we should be getting Discord Webhook notifications, as shown below:



Unblocking Blocked IP Address

We can also unblock the blocked ip using the green button with unblock ip text within. And discord webhook will also give us notifications if the ip is unblocked. As shown in figure below:



Conclusion and Suggestion

1. Conclusion

This project successfully developed a real-time network intrusion detection system targeting SMB-based attacks using Zeek as the core detection engine, integrated with a custom Python pipeline, Grafana for visualization, Loki for log aggregation, Discord for alerting, and an active response mechanism via iptables.

The system was able to detect all three stages of the simulated attack scenario; port scan reconnaissance, SMB share enumeration, and unauthorized file access. And respond to each stage with real-time dashboard updates, Discord notifications, and manual IP blocking through the Grafana interface.

2. Suggestion

Currently blocked IPs are stored in memory and lost when `block_agent.py` restarts. A future improvement would store the blocked IP list in a JSON file or SQLite database so blocks persist across restarts and can be audited later.

The current pipeline communicates over plain HTTP between the forwarder, Loki, and block agent. Adding TLS encryption would make the system suitable for deployment in environments where network security of the monitoring pipeline itself is a concern.

Requirements

Hardware:

- Laptop

Operating System:

- Ubuntu Server
- Kali Linux

Software:

- virt-manager
- Ms. Word
- Canva

File Information

No.	File Name	Description
1.	zeek_forwarder.py	Zeek log forwarder
2.	block_agent.py	Flask active response API
3.	detect-scan.zeek	Custom zeek port scan detection script
4.	local.zeek	Zeek site policy configuration
5.	node.cfg	Zeek node and interface configuration
6.	zeekctl.cfg	Zeek controller configuration
7.	dashboard.json	Grafana dashboard configuration
8.	loki-config.yaml	Loki configuration file
9.	Project Paper	This document
10.	Presentation	Presentation slide

